T-EDUCARE · PLATFORM EXPANSION

# New Module Specifications for UI/UX Design

Functional and experience requirements for four new modules — written for the design team to translate into wireframes and high-fidelity screens. Covers purpose, user roles, required screens, key flows, edge-case states, and design guidance for each module.

💰 Finance | 📊 Result Management | 🏠 Hostel Management | 📢 Announcements

**Audience**
UI/UX Design Team

**Prepared for**
T-Educare Product

**Status**
Ready for design exploration

## Contents

1. How to use this document

2. Finance Module
    1. Purpose & scope
    2. Roles & access
    3. Core screens
    4. Key flow — fee billing to payment
    5. States & edge cases
    6. Design notes
3. Result Management Module
4. Hostel Management Module
5. Announcement Module
6. Cross-module design principles

## How to use this document

### This is a functional brief, not a visual spec

Each module below describes **what needs to exist on screen and why** — the roles who use it, the screens required, the information each screen must surface, the flow a user moves through, and the states a screen can be in (empty, pending, error, locked, etc.). Visual direction, layout, and component choices are left to design judgment, but notes are included at the end of each module where a particular interaction pattern or precedent is strongly recommended. All four modules should inherit T-Educare's existing visual language (the shadcn-based design system already used across Dashboard, Staff Management, Lecture Management, and Attendance) so they read as one coherent product, not four bolted-on tools.

MODULE 1 OF 4

# Finance

A single source of truth for every money movement in the institution: student tuition and fees, outstanding balances, lecturer and staff salaries, and a full payment trail so every party — bursar, staff, and student — can see exactly where their records stand. This is the module most comparable to a banking/accounting product in tone: it needs to feel precise, trustworthy, and auditable.

## Roles & Access

**Institution Admin / Bursar**

Full access — configures fee structures, runs payroll, reconciles payments, views every record, generates reports.

**Staff / Lecturer**

Self-service only — views their own salary history and downloads payslips. Cannot see anyone else's figures.

**Student**

Self-service only — views their own fee bill and balance, pays via a third-party gateway, downloads receipts.

**Auditor view (future-ready)**

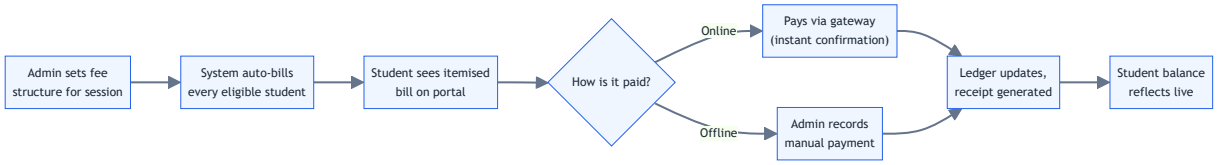
Read-only access to the transaction log and reports — worth designing the permission model so this role can be added later without rework.

## Core Screens

| SCREEN | PURPOSE & KEY ELEMENTS |
|---|---|
| **Finance Dashboard** | Bursar's at-a-glance view: total revenue this term, total outstanding, total payroll due, fee collection rate. Include an **aging breakdown of outstanding fees** (0–30 / 31–60 / 61–90+ days overdue) — the single most useful "who do we chase first" view in any accounts-receivable tool. A revenue trend chart and a "top outstanding debtors" shortlist round this out. |
| **Fee Structure Setup** | Admin defines fee types (Tuition, Hostel, Library, Development Levy, etc.), scoped per programme / level / session, with amount, due date, and an optional late-payment penalty rule (flat fee or percentage after the due date). |
| **Student Fee Ledger** | Institution-wide table: every student with total billed, total paid, balance, and a status pill ( Fully Paid Partial Owing ). Filterable by department/level/session. Drilling into one student opens an itemised ledger — every fee line, every payment against it, running balance. |
| **Record / Reconcile Payment** | Admin manually logs an offline payment (bank transfer, cash) with a reference number and proof upload, or reviews an incoming gateway payment and allocates it against the right fee lines if a student owes across multiple fee types. |
| **Payroll** | List of staff with salary, allowances, deductions, and monthly payment status. A "Run Payroll" action calculates net pay for the period and generates payslips in bulk. |
| **My Payslip (staff self-service)** | Monthly breakdown — basic pay, allowances, deductions, net pay — with history by month and a PDF download. |

| | |
|---|---|
| **Student Payment Portal** | The student-facing "pay now" screen: current bill broken down by fee type, a clear total, and a payment button that hands off to a third-party gateway (Paystack/Flutterwave-style). Needs to read as trustworthy as a banking checkout — this is where real money changes hands. |
| **Transaction / Audit Log** | An immutable, chronological log of every financial event — who recorded it, exactly when, the amount, and a reference. This is the record a dispute gets resolved against, so it should never be editable from the UI. |
| **Reports & Export** | Generate and export (Excel/PDF) termly revenue reports, outstanding-fee reports, and payroll summaries for offline use by the bursar's office or auditors. |

## Key Flow — Billing to Payment




## States & Edge Cases

**Partial payment** balance carries forward, visibly flagged.

**Overpayment** results in a credit balance, shown distinctly from "owing."

**Failed gateway transaction** student needs a clear retry path, not a dead end.

**Waived / discounted fee** e.g. scholarship — shown as a line item, not a silent balance change.

**Late penalty applied** auto-added after due date, shown as its own line with the rule that triggered it.

**Empty states** no fee structure configured yet; no transactions yet for a new session.

### 💡 Design notes

- Use one consistent colour code for payment status everywhere in the module — green/amber/red for Paid/Partial/Owing — so a bursar can scan a table and triage in seconds.
- The student payment screen is the one place in the whole platform where trust signals matter most: clear totals, a lock/secure icon, recognisable gateway logos, and a receipt immediately after payment.
- Design the dashboard's aging buckets the way accounting software does — it's an unusual but very high-value view for a bursar, worth the extra design effort.
- This flow will be used heavily on mobile by parents and students paying fees away from a desk — the payment screen specifically should be designed mobile-first.

MODULE 2 OF 4

# Result Management

Manages the full lifecycle of a student's result — upload, review, approval, and publication — with one non-negotiable rule: **once a result is approved and published, it can never be silently overwritten**. Every correction after that point is recorded as a new, reasoned update that supersedes the old one without erasing it. Deadlines and penalties for late submission are admin-configurable, and results lock into read-only history once a session rolls over.

## Roles & Access

**Lecturer**
Uploads and updates results for the courses they teach. Can submit corrections post-publish, but never edit a published record directly.

**Exam Officer / HOD (optional review tier)**
Reviews a submitted result batch before it reaches final admin approval.

**Institution Admin / Registrar**
Gives final approval and publishes. Sets deadlines and penalty rules.

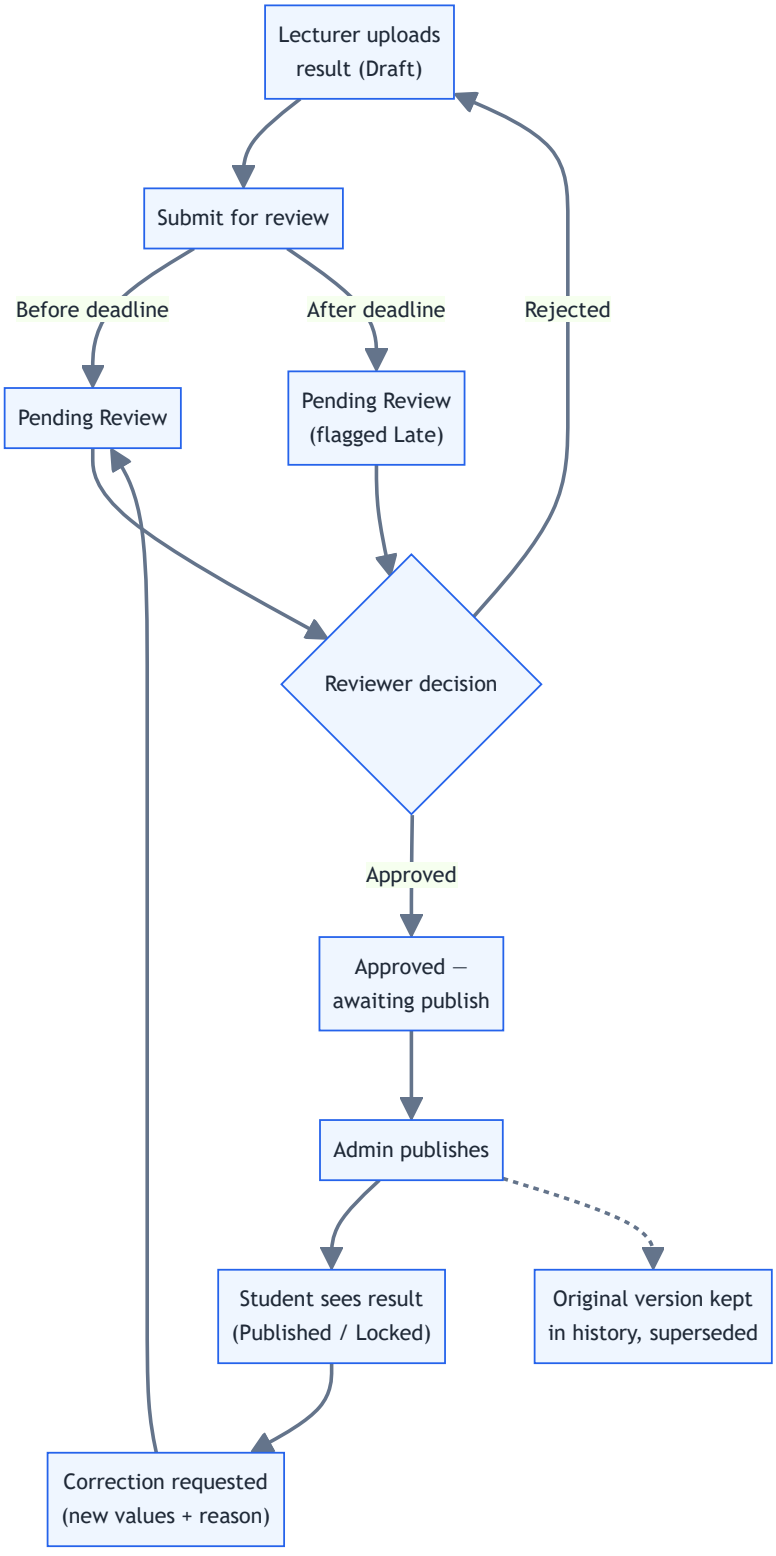
**Student**
Sees only published results for themselves — never a draft, never another student's.

## Core Screens

| SCREEN | PURPOSE & KEY ELEMENTS |
|---|---|
| **Result Upload (Lecturer)** | Select course + session/semester + level, then either bulk-upload a CSV score sheet or enter scores in a grid (CA score, exam score, computed total/grade per student). Saves as a draft until submitted. |
| **Deadline & Penalty Settings (Admin)** | Set a result-upload deadline per session/semester (globally or per course), and a penalty rule for late submission — a visible flag on the batch, a grace-period countdown, or a formal escalation notice to the admin. |
| **Pending Review Queue** | Every submitted-but-not-yet-approved batch, with lecturer, course, submission date, and a late-submission flag if applicable. |
| **Review & Approval** | Reviewer sees the full score grid and either approves it (moves to publish-ready) or rejects with a required comment, sending it back to the lecturer for correction. |
| **Publish** | The point of no casual return. Admin publishes an approved batch; students can now see it. This action should feel deliberate in the UI — a confirmation step, not a casual button. |
| **Result Version History** | For any result record: a timeline of every version — the original upload (who, when) and every subsequent correction (who, when, and the **mandatory reason** given), with the current version clearly marked. Nothing is ever deleted from this timeline. |

| | |
|---|---|
| **Post-Publish Correction** | Initiating a change to an already-published result opens the same "new values + required reason" form, and the correction goes through the same review/approval cycle again before it becomes the new current version. The old published version remains visible in history, clearly labelled as superseded. |
| **Student Result View / Transcript** | Published results only, grouped by session/semester, with GPA per semester and a running CGPA. Needs to look and print like a real academic transcript — this gets used for NYSC, job, and further-study applications. |

## Key Flow — Upload to Publish, and Correction




## States & Edge Cases

**Draft** only the lecturer can see it.

**Submitted / Under review** visible to reviewer only.

**Approved, unpublished** exists but students can't see it yet.

**Published / Locked** visible to the student; structurally not directly editable.

**Superseded** an older published version, kept for history only.

**Archived (post-rollover)** a prior session's results, fully read-only.

### 💡 Design notes

- Make "Published — Locked" visually unmistakable (a padlock, a distinct colour, a persistent label) — the goal is for tampering to feel *structurally* impossible in the interface, not just policy-forbidden.
- The version-history screen is the most novel part of this module — design it like a document revision timeline (similar to a Git commit log or a Google Docs version history): vertical timeline, each entry a card with timestamp, actor, reason, and a highlighted diff of what changed (e.g. "CA Score: 18 → 20").
- Bulk CSV upload needs real inline validation before submit — a downloadable template, and specific per-row errors ("Row 23: Student ID not found") rather than a single generic failure message.
- Give lecturers a visible deadline countdown that shifts from neutral to amber to red as the cutoff approaches — the same urgency pattern a submission portal would use.
- The transcript view is a document people will print and submit externally — design it to look authoritative on paper, not just on screen.

MODULE 3 OF 4

# Hostel Management

Manages hostel, room, and bed inventory with a genuinely visual browsing experience — students should be able to see what a hostel and a specific room actually look like before applying, the same way they'd browse a hotel-booking site. Admins define the inventory, open an application window, process applications, and allocate beds; students then complete payment through the Finance module via a third-party gateway.

## Roles & Access

**Institution Admin**

Sets up hostels/rooms/beds, opens and closes the application window, processes and allocates applications.

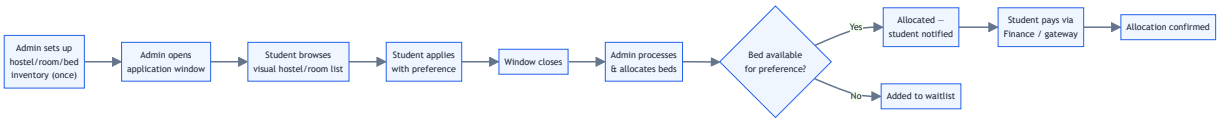
**Student**

Browses available hostels/rooms, applies with a preference, tracks allocation status, pays once allocated.

## Core Screens

| SCREEN | PURPOSE & KEY ELEMENTS |
|---|---|
| **Hostel Inventory Setup** | Admin creates a Hostel (name, gender restriction, description, photos), then adds Rooms within it (room number, bed capacity — e.g. 4 per room, amenities), with each bed individually trackable as occupied or vacant. |
| **Hostel & Room Browser** | A visual, listing-style browse experience for students (and usable by admin for oversight): a grid of hostels with a photo and vacancy summary; clicking in shows the rooms, and clicking a room shows its detail — capacity, current occupancy ("2 of 4 beds taken"), amenities, photos. |
| **Application Window Settings** | Admin sets the application open and close dates for the session, which hostels/rooms are available, and any eligibility rule (e.g. only returning students, or a specific level). |
| **Student Application** | Student browses only rooms with vacancy, selects a preferred hostel/room (optionally ranks 2–3 choices), and submits. Status becomes "Application Submitted — Pending Allocation." |
| **Admin Allocation Dashboard** | Every application, filterable by preference/gender/level, with an allocate action that assigns a specific bed. An optional auto-suggest (best-fit by preference and availability) speeds up bulk processing, with manual override always available. |
| **My Hostel (student self-service)** | Once allocated: assigned hostel, room, and bed, a downloadable allocation letter, and a "Proceed to Payment" action that hands off into the Finance module's payment gateway. |
| **Occupancy Overview** | Admin's live map of every room across every hostel, colour-coded by fill level — the fastest way to see capacity at a glance across the whole institution. |
| **Waitlist** | Students who applied but couldn't be placed (hostel full) — admin manages this list and reassigns when a bed frees up, e.g. after a cancellation. |

## Key Flow — Application to Allocation




## States & Edge Cases

**Room fills while browsing** near-real-time vacancy so students don't apply for a full room.

**Payment not completed** define a grace period after which an unpaid allocation can be released back to the pool.

**Application window closed** read-only "Applications closed" state, no apply action available.

**Gender-restricted hostel** enforced before a student can even view/apply.

**Cancellation / room-change request** a student-initiated flow back into the allocation/waitlist process.

**Empty states** no hostels configured yet; application window not yet open.

### 💡 Design notes

- This is the most visually rich module of the four — lean into it like a hotel/short-let booking experience: hero images for each hostel, room cards with an occupancy progress bar ("3 of 4 beds filled"), and ideally a simple floor-plan-style grid of room icons colour-coded by vacancy.
- Give the student application a clear stepper (Browse → Select → Apply → Await Allocation → Pay → Confirmed) — housing is something students are genuinely anxious about, and a visible process reduces that.
- The admin's occupancy heat-map is a high-value "glanceable" screen, the same idea as Finance's aging-bucket dashboard — worth the same level of design attention.
- Once a room is allocated, consider showing bed-level avatars/initials so an admin can visually audit who's sharing a room at a glance.

MODULE 4 OF 4

# Announcements

A flexible broadcast tool: compose one announcement and target it precisely — everyone, a single department, a specific level (e.g. 100L), staff only, or hand-picked individuals — then deliver it through the channel that fits (in-app, email, or both). Flexibility in targeting is the whole point of this module.

## Roles & Access

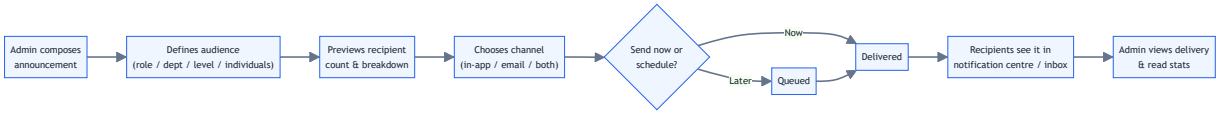
**Admin / permitted staff**
Composes and sends announcements, defines the audience and channel.

**Student / Staff (recipient)**
Receives via the chosen channel(s) and reads it in an in-app notification centre.

## Core Screens

| SCREEN | PURPOSE & KEY ELEMENTS |
|---|---|
| **Compose Announcement** | Title, rich-text body (with attachment/image support), and a targeting panel: audience type (Everyone / Students only / Staff only / specific Department(s) / specific Level(s) / hand-picked individuals via search), with filters combinable (e.g. "100L students in Computer Science"). Delivery channel choice: In-App only / Email only / Both. Option to send now or schedule for later. |
| **Audience Preview** | Before sending: "This will reach approximately 342 recipients," with a breakdown (e.g. 300 students, 42 staff) and an expandable list — so nothing is ever sent blind. |
| **Sent History** | Every past announcement — title, audience summary, channel, sent date, and delivery stats (e.g. "298/300 emails delivered," "150 read in-app"). |
| **Drafts & Scheduled** | Manage not-yet-sent announcements — edit a draft, or cancel/reschedule one that's queued. |
| **Notification Centre (recipient)** | In-app feed of received announcements, with a clear unread/read state, grouped by date (Today / Yesterday / This week). |
| **Announcement Detail** | Full content view when a recipient opens one, which also marks it as read for the sender's delivery stats. |

## Key Flow — Compose to Delivery



## States & Edge Cases

**Zero matching recipients** warn before send rather than silently sending nothing.

**Partial delivery failure** some emails bounce — reflected in delivery stats, not hidden.

**Very large audience** an async "sending…" progress state rather than a blocking spinner.

**Draft auto-save** composing shouldn't risk losing work.

**Cancel a scheduled send** must be possible any time before it fires.

### 💡 Design notes

- The audience-targeting UI is the heart of this module — design it as a clean, filter-chip based builder (in the spirit of an email-marketing segment builder) rather than a dense form, so the flexibility doesn't feel like complexity.
- Give announcements a consistent category/urgency tag (General, Urgent, Academic, Event) with colour coding, carried through from compose into the recipient's inbox view.
- The recipient-facing notification centre should feel like a lightweight inbox — bold/dot unread indicators, grouped by date, quick to scan.
- Consider an optional "Urgent" flag that also triggers a prominent banner on the recipient's next login, not just a quiet inbox entry — useful for time-sensitive notices (exam changes, closures).

## Cross-Module Design Principles

A few things worth holding constant across all four modules so they feel like one coherent product rather than four separate tools bolted onto the platform.

**01**

### Inherit the existing design system

All four modules sit inside the same shadcn-based system already used across Dashboard, Staff Management, Lecture Management, and Attendance — same typography, spacing, table and card patterns, and component library. New screens should feel native, not imported.

**02**

### Consistent status colour language

Green/amber/red (success, pending/attention, problem) should mean the same thing everywhere — a paid fee, an on-time result upload, and an allocated hostel bed are all "green" states; keep that vocabulary consistent module to module.

**03**

### Design every empty, loading, and error state

Each module has a genuine "nothing here yet" moment (no fees configured, no results uploaded, no hostels set up, no announcements sent) — these need real designed states, not blank screens, since institutions will see them on day one.

**04**

### Self-service views are first-class, not afterthoughts

The student/staff-facing screens (payment portal, payslip, my hostel, result transcript, notification centre) are what the majority of users will actually touch day to day — they deserve the same design care as the admin tooling, not a stripped-down leftover.

**05**

### Mobile-first where money or urgency is involved

Fee payment, result viewing, and urgent announcements are all things students and parents will act on from a phone, often away from a desk — these specific flows should be designed mobile-first, not adapted after the fact.

**06**

### Make irreversible actions feel irreversible

Publishing a result, sending an announcement to the whole institution, and finalising a hostel allocation are all hard-to-undo actions — each should have a deliberate confirmation step in the UI that matches the real-world weight of the action.